Single-Cycle RISC-V Processor in Verilog

Complete Learning Guide, Code, Viva Preparation and Project Report

Part 1 — Project Overview
- What we built
- Why these 8 instructions?
- Project files
- The one sentence to remember

Part 2 — Simplified Architecture (from zero)
- 2.1 What is RISC-V?
- 2.2 What does “single-cycle” mean?
- 2.3 What happens when the processor executes one instruction?
- 2.4 The parts, one by one
- 2.5 The whole thing as a story

Part 3 — Datapath Diagram
- 3.1 Full diagram
- 3.2 Simple version to draw on the board
- 3.3 Every arrow explained
- 3.4 Two things happen at the clock edge

Part 4 — Instruction Subset
- 4.1 The 8 instructions
- 4.2 How the 32 bits are arranged (instruction formats)
- 4.3 Opcodes and function codes
- 4.4 Control signals for every instruction (the “truth table”)
- 4.5 Decoding a real instruction by hand

Part 5 — Verilog Basics and the Complete Code
- 5.1 Software vs hardware description (read this first!)
- 5.2 The keywords you need (and nothing more)
- 5.3 Module: `program_counter`
- 5.4 Module: `instruction_memory`
- 5.5 Module: `register_file`
- 5.6 Module: `alu`
- 5.7 Module: `control_unit`
- 5.8 Module: `imm_gen`
- 5.9 Module: `data_memory`
- 5.10 Module: `riscv_cpu` (top level)

Part 6 — Testbench
- 6.1 Background concepts
- 6.2 The code
- 6.3 Block by block
- 6.4 How do I know my processor is working?

Part 7 — Example Program
- 7.1 The program
- 7.2 Step-by-step execution
- 7.3 Final state

Part 8 — Simulation Instructions
- 8.1 Install
- 8.2 Create the files
- 8.3 Compile
- 8.4 Run
- 8.5 View waveforms
- 8.6 Common problems

Part 9 — Expected Results

Part 10 — Detailed Explanation: Every Instruction Inside the Processor
- 10.1 ADD — `add x3, x1, x2` (x3 = x1 + x2)
- 10.2 SUB — `sub x4, x1, x2` (x4 = x1 − x2)
- 10.3 AND — `and x5, x1, x2`

10.4 OR — `or x6, x1, x2`
10.5 ADDI — `addi x1, x0, 12` (x1 = x0 + 12)
10.6 LW — `lw x7, 0(x0)` (x7 = Memory[x0 + 0])
10.7 SW — `sw x3, 0(x0)` (Memory[x0 + 0] = x3)
10.8 BEQ — `beq x3, x7, 8` (if x3 == x7, PC = PC + 8)
10.9 Summary: which parts each instruction uses

Part 11 — Viva Questions and Answers

11.1 Basics
11.2 Components
11.3 Instruction working
11.4 Clock, reset, Verilog, testing
11.5 Limitations and future scope
11.6 Follow-up questions a professor might ask

Part 12 — Presentation: 10 Slides with Speaking Notes

Part 13 — One-Day Learning Plan

Part 14 — Project Report

Abstract
Chapter 1 — Introduction
Chapter 2 — Single-Cycle Processor
Chapter 3 — Architecture
Chapter 4 — Instruction Set
Chapter 5 — Verilog Design
Chapter 6 — Datapath
Chapter 7 — Example Execution
Chapter 8 — Simulation
Chapter 9 — Limitations
Chapter 10 — Future Scope
Chapter 11 — Viva Questions (Summary)
Conclusion
References

What this document gives you, in order

1. Project overview
2. Simplified architecture (explained from zero)
3. Datapath diagram
4. Instruction subset
5. Verilog basics + complete Verilog code (every module explained)
6. Testbench
7. Example program
8. Simulation instructions
9. Expected results
10. Detailed explanation: every instruction, step by step
11. Viva questions + answers
12. Presentation slides + speaking notes
13. One-day learning plan
14. Project report (Chapters 1–11, ready to submit)

The design was actually compiled and simulated with Icarus Verilog. All 11 checks pass. The output you see in Part 9 is the real output.

Part 1 — Project Overview

What we built

A small processor, written in Verilog, that can run real RISC-V machine code. It supports **8 instructions**:

`ADD`, `SUB`, `AND`, `OR`, `ADDI`, `LW`, `SW`, `BEQ`

It is **single-cycle**: every instruction is completed in exactly one tick of the clock.

Why these 8 instructions?

They are the smallest set that still shows every important idea of a processor:

Idea we want to show	Instruction that shows it
Doing maths with two registers	ADD, SUB
Doing logic with two registers	AND, OR
Using a constant number inside the instruction	ADDI
Reading from memory	LW
Writing to memory	SW
Making a decision / jumping	BEQ

If we removed any one group, we would lose an important concept. If we added more (shifts, multiply, jumps, etc.), the processor would get bigger without teaching anything new.

Project files

File	What it is	Lines (approx.)
`program_counter.v`	Holds the address of the current instruction	20
`instruction_memory.v`	Stores the program	40
`register_file.v`	The 32 registers x0–x31	40
`alu.v`	Does add / subtract / and / or	35
`control_unit.v`	Decodes the instruction, gives orders	80
`imm_gen.v`	Pulls the constant number out of the instruction	35
`data_memory.v`	RAM for LW / SW	35
`riscv_cpu.v`	Top module: wires everything together (+ 3 muxes)	100
`testbench.v`	Runs the CPU and checks the answers	85

The whole processor is about **400 lines** of simple Verilog. No SystemVerilog, no advanced tricks.

The one sentence to remember

> **Every clock cycle: the PC points to an instruction → we fetch it → decode it → read registers → ALU calculates → (maybe) use memory → write the result back → choose the next PC.**

If you can explain this sentence with the diagram in Part 3, you can handle most of the viva.

Part 2 — Simplified Architecture (from zero)

2.1 What is RISC-V?

A processor only understands a fixed list of commands, called its **instruction set** (ISA). Intel/AMD chips use the x86 instruction set. Phone chips use ARM.

RISC-V (pronounced “risk-five”) is another instruction set, with three things that make it great for students:

- It is **open and free**. Anyone can build a RISC-V processor without paying a licence fee.
- It is **simple and regular**. Every instruction is 32 bits, and the fields (register numbers, opcode) are in the same places in most instructions.
- It is **real**. It is used in actual chips today, not just in textbooks.

RISC means *Reduced Instruction Set Computer*: use a small number of simple instructions, each doing one small job, instead of many complicated instructions. The “V” means it is the fifth RISC design from UC Berkeley.

2.2 What does “single-cycle” mean?

The processor has a clock — a signal that goes tick, tick, tick. **Single-cycle** means that **one whole instruction is finished in one clock tick**: fetching, decoding, calculating, memory, and saving the result all happen in that one cycle.

Analogy: imagine a one-person shop. The shopkeeper takes an order, finds the item, bills it, and hands it over — completely — before the next customer steps up. That is single-cycle. (A **pipelined** processor is like an assembly line where different workers handle different steps of different customers at the same time. We are *not* doing that.)

The good part: very easy to understand. The bad part: the clock must be slow enough for the *slowest* instruction (LW, which uses everything), so every instruction takes that long.

2.3 What happens when the processor executes one instruction?

Five steps, all inside one clock cycle:

Step	Name	Question it answers
1	**Fetch**	“Which instruction is next? Give it to me.”
2	**Decode**	“What kind of instruction is this? Which registers? What constant?”
3	**Execute**	“Do the calculation.” (ALU)
4	**Memory**	“Do I need to read or write RAM?” (only LW / SW)
5	**Write back**	“Save the answer into the destination register.”

Then the **next PC** is chosen, and on the next clock tick everything repeats.

2.4 The parts, one by one

Program Counter (PC)

A 32-bit register holding the **address of the current instruction**.

Analogy: a bookmark in a recipe book — “I’m on step 3.”

Each instruction is 4 bytes long, so normally the next one is at `PC + 4`. After reset, PC = 0, so the program starts at address 0.

Instruction Memory

Stores the program as 32-bit machine-code numbers.

Analogy: the recipe book itself. The PC says “page 8”, the book shows what’s written on page 8.

Input: address. Output: the instruction at that address. The processor only *reads* it.

Register File

32 small, very fast storage boxes called **registers**, named `x0` to `x31`. Each holds one 32-bit number.

Analogy: 32 labelled boxes on your desk. RAM is the cupboard in another room — slower to reach. Registers are right there.

- It can **read two registers at once** (because instructions like `ADD x3, x1, x2` need two inputs).
- It can **write one register** per cycle (the answer).
- **`x0` is always 0.** Writing to it does nothing. This is a RISC-V rule and is very handy (e.g. `ADDI x1, x0, 12` means "x1 = 0 + 12").

ALU (Arithmetic Logic Unit)

The calculator. Takes two numbers, does one operation, gives one result. Ours does **add, subtract, AND, OR**.

It also outputs a 1-bit signal **`zero`**, which is 1 when the result is 0. BEQ uses this: if `a − b = 0` then `a == b`.

Data Memory

The processor's RAM, where variables live.

- `LW` (load word): read a value from memory into a register.
- `SW` (store word): copy a register into memory.

Analogy: the cupboard. You can take something out (LW) or put something in (SW).

Control Unit

The **brain / manager**. It looks at the instruction's **opcode** (the last 7 bits) and decides what everyone else should do. It produces a handful of yes/no signals:

Signal	Meaning when it is 1
`reg_write`	Save a result into register `rd`
`alu_src`	ALU's 2nd input is the **immediate** (not register rs2)
`mem_write`	Write into data memory (SW)
`mem_to_reg`	The value to save comes from **memory** (LW), not the ALU
`branch`	This is a BEQ
`alu_ctrl` (3 bits)	Which ALU operation: 000 add, 001 sub, 010 and, 011 or

Analogy: a traffic police officer at a junction. The roads (wires) are always there; the officer decides which way the traffic goes for this particular car.

Multiplexers (MUX)

A MUX is a **selector switch**: two inputs, one output, and a select signal that picks which input goes through.

```
sel = 0  -> output = input0
sel = 1  -> output = input1
```

Why we need them: different instructions need data to flow different ways.

- `ADD` wants the ALU's second input to be a register. `ADDI` wants it to be a constant. → **MUX 1** (controlled by `alu_src`)
- `ADD` saves the ALU result. `LW` saves the value read from memory. → **MUX 2** (controlled by `mem_to_reg`)
- Normally the next PC is `PC + 4`. For a taken BEQ, it's `PC + offset`. → **MUX 3** (controlled by `branch AND zero`)

We have exactly **three** muxes. That's it.

Immediate value and the Immediate Generator

An **immediate** is a constant number written *inside* the instruction itself. In `ADDI x1, x0, 12`, the `12` is the immediate. In `LW x7, 0(x0)`, the `0` is the immediate (offset).

Different instruction types store this number in different bit positions (to keep register fields in the same place). The **Immediate Generator** collects those bits and builds a full 32-bit number. It also does **sign extension**: if the constant is negative, it fills the upper bits with 1s so the value stays negative.

How is the next PC decided?

```
if (instruction is BEQ) AND (the two registers were equal)
        next PC = PC + immediate     (jump)
else
        next PC = PC + 4             (go to the next instruction)
```

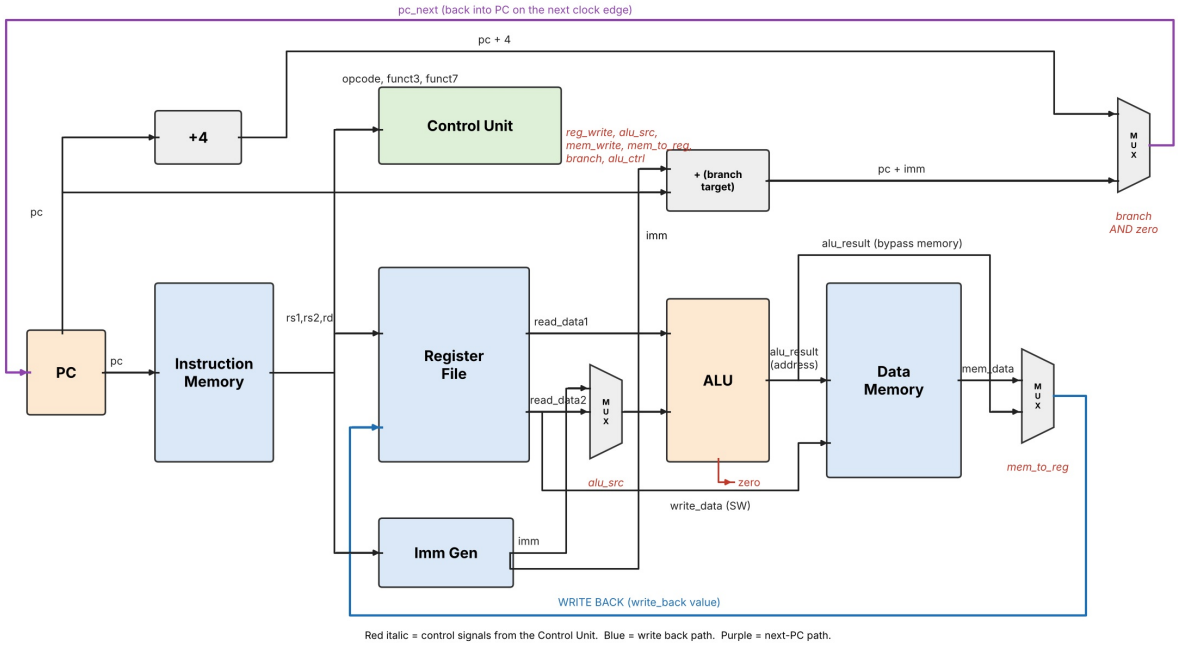
That's all. This is "branch logic".

2.5 The whole thing as a story

> The **PC** says "address 8". The **Instruction Memory** hands over the instruction at 8: `ADD x3, x1, x2`. The **Control Unit** reads the opcode and says "this is R-type ADD: ALU should add, use a register as the 2nd input, save the result, don't touch memory, don't branch." The **Register File** gives out x1 = 12 and x2 = 10. The **ALU** adds them: 22. The memory is ignored. The write-back MUX picks the ALU result, and at the clock edge 22 is saved into x3. Meanwhile PC + 4 = 12 was calculated and also goes into the PC at the same clock edge. Next cycle, we start again at 12.

Part 3 — Datapath Diagram

3.1 Full diagram



Datapath of the single-cycle processor

3.2 Simple version to draw on the board

If you only have one minute and a marker, draw this:

```
          +---------------------- next PC MUX <---- PC+4
          |                        ^      <---- PC+imm (BEQ)
          v                        |
       +----+    +-------+    +----------+    +-----+    +--------+    +-----+
clk -->| PC |--->| Instr |--->| Register |--->| MUX |--->|  ALU   |--->| Data|
       +----+    |  Mem  |    |   File   |    +-----+    +--------+    | Mem |
                 +-------+    +----------+       ^           |  zero   +-----+
                     |            ^              |           |            |
                     v            |          Imm Gen        v            v
                Control Unit      |                    (alu_result)   (mem_data)
                (sends signals    |                          \         /
                 to everything)   |                           \       /
                                  +-------- write-back MUX <----+----+
```

3.3 Every arrow explained

Arrow	From → To	What travels on it	When is it used?
pc	PC → Instruction Memory	Address of current instruction	Always
instr	Instruction Memory → everyone	The 32-bit instruction, split into fields	Always
opcode, funct3, funct7	Instruction → Control Unit	Tells control which instruction it is	Always
rs1, rs2, rd	Instruction → Register File	Which registers to read and which to write	Always
instr	Instruction → Imm Gen	Bits containing the constant	ADDI, LW, SW, BEQ
read_data1	Register File → ALU input A	Value of rs1	Always
read_data2	Register File → MUX 1 and → Data Memory	Value of rs2	R-type, BEQ (ALU); SW (memory)
imm	Imm Gen → MUX 1 and → branch adder	32-bit constant	ADDI, LW, SW (ALU); BEQ (adder)
alu_b	MUX 1 → ALU input B	rs2 value or immediate	Always
alu_result	ALU → Data Memory address and → MUX 2	Answer, or memory address	Always
zero	ALU → next-PC MUX	1 if ALU result is 0	BEQ
mem_data	Data Memory → MUX 2	Value read from RAM	LW
write_back	MUX 2 → Register File	Value to save into rd	When reg_write = 1
pc + 4	Adder → next-PC MUX	Address of next instruction	Normal case
pc + imm	Adder → next-PC MUX	Branch target address	BEQ taken
pc_next	next-PC MUX → PC	New PC, loaded at clock edge	Always

Important idea: in hardware, *all* arrows carry values *all the time*. The control signals just decide which values are actually *used*. For example, during `ADD`, the data memory is still being given an address — but `mem_write = 0` so nothing is written, and `mem_to_reg = 0` so its output is ignored.

3.4 Two things happen at the clock edge

Everything in the diagram is “instant” (combinational) **except three things**, which only change when the clock rises:

1. The **PC** loads `pc_next`.
2. The **Register File** writes `rd` (if `reg_write = 1`).
3. The **Data Memory** writes (if `mem_write = 1`).

Between clock edges, values flow through the wires and settle. At the edge, the results get “locked in”. That is the heart of single-cycle design.

Part 4 — Instruction Subset

4.1 The 8 instructions

Instruction	Example	Meaning	Type
ADD	`add x3, x1, x2`	x3 = x1 + x2	R
SUB	`sub x4, x1, x2`	x4 = x1 − x2	R
AND	`and x5, x1, x2`	x5 = x1 AND x2 (bit by bit)	R
OR	`or x6, x1, x2`	x6 = x1 OR x2 (bit by bit)	R
ADDI	`addi x1, x0, 12`	x1 = x0 + 12	I
LW	`lw x7, 0(x0)`	x7 = Memory[x0 + 0]	I
SW	`sw x3, 0(x0)`	Memory[x0 + 0] = x3	S
BEQ	`beq x3, x7, 8`	if (x3 == x7) PC = PC + 8	B

4.2 How the 32 bits are arranged (instruction formats)

Bit 31 is on the left, bit 0 on the right.

R-type (ADD, SUB, AND, OR) — two source registers, one destination:

funct7 [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

I-type (ADDI, LW) — one source register, one constant, one destination:

imm[11:0] [31:20]	rs1 [19:15]	funct3 [14:12]	rd [11:7]	opcode [6:0]

S-type (SW) — no destination register, so the constant is split into two pieces:

imm[11:5] [31:25]	rs2 [24:20]	rs1 [19:15]	funct3 [14:12]	imm[4:0] [11:7]	opcode [6:0]

B-type (BEQ) — like S-type but the bits are shuffled, and the offset is always even:

imm[12, 10:5] [31:25]	rs2	rs1	funct3	imm[4:1, 11] [11:7]	opcode

Why is the immediate split up? So that **rs1, rs2 and rd are always in the same bit positions** in every format. The register file can then start reading registers before the control unit has even finished deciding what the instruction is. Simpler, faster hardware.

4.3 Opcodes and function codes

Instruction	opcode	funct3	funct7 (bit 30)
ADD	0110011	000	0
SUB	0110011	000	**1**
AND	0110011	111	0
OR	0110011	110	0
ADDI	0010011	000	—
LW	0000011	010	—
SW	0100011	010	—
BEQ	1100011	000	—

Notice ADD and SUB have the **same** opcode and funct3. Only bit 30 tells them apart. That's why our control unit looks at `instr[30]`.

4.4 Control signals for every instruction (the “truth table”)

This table is the entire control unit. Learn it — it is the most likely thing to be asked.

Instr	reg_write	alu_src	mem_write	mem_to_reg	branch	alu_ctrl
ADD	1	0 (rs2)	0	0 (ALU)	0	000 (add)
SUB	1	0	0	0	0	001 (sub)
AND	1	0	0	0	0	010 (and)
OR	1	0	0	0	0	011 (or)
ADDI	1	**1** (imm)	0	0	0	000 (add)
LW	1	1	0	**1** (mem)	0	000 (add)
SW	**0**	1	**1**	0 (don’t care)	0	000 (add)
BEQ	**0**	0	0	0 (don’t care)	**1**	001 (sub)

Quick patterns to remember:

- **Only SW and BEQ don’t write a register** (they have no rd).
- **alu_src = 1 for anything with a constant used by the ALU**: ADDI, LW, SW.
- **LW, SW, ADDI all use ADD** in the ALU (LW/SW to compute the address).
- **BEQ uses SUB** — if the result is zero, the two registers were equal.

4.5 Decoding a real instruction by hand

`add x3, x1, x2` is stored as hex `002081b3`. In binary:

```
0000000 00010 00001 000 00011 0110011
funct7   rs2   rs1  f3   rd   opcode
  0       2     1   ADD  3    R-type
```

So: R-type, funct3 = 000 and bit 30 = 0 → ADD; read x1 and x2; write x3. Exactly what we wanted.

`sub x4, x1, x2` is `40208233`:

```
0100000 00010 00001 000 00100 0110011
   ^ bit 30 = 1  → SUB
```

Part 5 — Verilog Basics and the Complete Code

5.1 Software vs hardware description (read this first!)

In C/Python/Java you write **steps**. The computer does step 1, then step 2, then step 3.

In Verilog you **describe a circuit**: which boxes exist and how wires connect them. A circuit has no "step 1 then step 2". **All parts work at the same time, all the time.** If you write two `assign` lines, they are two separate pieces of hardware sitting side by side, both active forever.

Software (C, Python)	Hardware description (Verilog)
Lines run one after another	Everything runs **in parallel**, all the time
A variable is a memory location	A `wire` is a physical connection; a `reg` is a storage element or a value computed by a block
A function is called when needed	A `module` is a **physical block** that always exists
Calling a function twice = run code twice	Using a module twice = **two copies of the hardware**
Time doesn't matter much	Time matters: things change at **clock edges**

Mental picture: writing Verilog is like drawing a circuit diagram using text. The simulator then pretends to be that circuit.

5.2 The keywords you need (and nothing more)

`module` ... `endmodule`

A module is one **box** in the diagram — like a chip with pins.

```
module alu ( ...pins... );
    ...what is inside...
endmodule
```

`input` and `output`

The **pins** of the box. `input` = data coming in, `output` = data going out.

Bit widths: `[31:0]`

`[31:0]` means "32 wires, numbered 31 down to 0" — a 32-bit number. `[4:0]` is 5 bits (enough for a register number 0–31). With no width, it is 1 bit.

You can pick out bits: `instr[6:0]` = the lowest 7 bits of `instr` (the opcode). `instr[30]` = just bit 30.

Writing numbers: `32'd0`, `3'b001`, `32'h00c00093`

Format: `<how many bits>'<base><value>`.

- `32'd4` = the number 4, written in **d**ecimal, 32 bits wide
- `3'b001` = **b**inary 001, 3 bits wide
- `32'h00c00093` = **h**exadecimal, 32 bits wide

`wire`

A plain connection. It has no memory — it just carries whatever is driven into it. Used with `assign` or to connect modules.

`reg`

A variable that gets its value inside an `always` block. **Beginner warning:** `reg` does *not* always mean a real hardware register. Inside `always @(*)` it is just combinational logic. Inside `always @(posedge clk)` it becomes a real storage element (flip-flop). The rule is simple: *if you assign it inside `always`, declare it `reg`; otherwise `wire`.*

`assign`

A permanent connection / formula.

```
assign zero = (result == 32'd0);
```

This means "`zero` is *always* equal to `(result == 0)`." Whenever `result` changes, `zero` changes immediately. It is not executed once — it is a piece of hardware that is always on.

The `? :` operator (this is our MUX)

```
assign alu_b = (alu_src == 1) ? imm : read_data2;
```

Read as: "if `alu_src` is 1, `alu_b` gets `imm`, otherwise `read_data2`." This single line **is** a 2-to-1 multiplexer. That's why we don't need a separate mux module.

`always @(*)` — combinational logic

"Whenever **any** input used inside changes, recompute." Used for things that have no memory: the ALU, the control unit, the immediate generator. The `*` means "watch everything I read".

```
always @(*) begin
    case (alu_ctrl)
        3'b000: result = a + b;
        ...
    endcase
end
```

`always @(posedge clk)` — clocked (sequential) logic

"Do this **only at the moment the clock goes from 0 to 1**." Used for things with memory: PC, register file writes, data memory writes. Between clock edges, the value stays the same.

`=` vs `<=`

- `=` (blocking) — use inside `always @(*)`.
- `<=` (non-blocking) — use inside `always @(posedge clk)`.

Just follow that rule. (Why: `<=` makes all the flip-flops update together at the clock edge, like real hardware.)

`case`

Like `switch` in C:

```
case (opcode)
    7'b0110011: begin ... end   // R-type
    7'b0010011: begin ... end   // ADDI
    default:    ...
endcase
```

`begin ... end`

Curly braces `{ }` of Verilog — they group several lines.

`if` / `else`

Same as C. Used inside `always` blocks.

Arrays (memories): `reg [31:0] mem [0:63];`

"64 items, each 32 bits." Access with `mem[5]`. This is how we build the register file and memories.

`initial`

Runs **once at the start of simulation**. We use it to fill memories with starting values and to drive the testbench. (In real chips `initial` doesn't exist; it is a simulation feature.)

`for` loops

Only used inside `initial` to fill arrays with zeros. In hardware descriptions, a `for` loop doesn't mean "repeat over time" — it's just a shortcut for writing 64 lines.

Module instantiation (placing a box and wiring its pins)

```
alu alu_unit (
    .a(read_data1),     // connect pin "a" of the ALU to wire read_data1
    .b(alu_b),
    .alu_ctrl(alu_ctrl),
    .result(alu_result),
    .zero(zero)
);
```

`alu` = which kind of box. `alu_unit` = the name of this particular copy. `.pin(wire)` = "plug this pin into this wire".

Concatenation `{ }` and replication `{N{ }}`

`{a, b}` glues bits together side by side. `{20{instr[31]}}` repeats bit 31 twenty times. We use both in the immediate generator.

`parameter`

A named constant (like `#define` in C). **We deliberately don't use any** — every size is written directly so you can see the real numbers. If asked: "a parameter lets you change a size in one place; we kept fixed sizes for simplicity."

5.3 Module: `program_counter`

Purpose: remember the address of the current instruction, and update it every clock tick.

Inputs: `clk` (clock), `reset` (1 = go back to 0), `pc_next` (address to load next). **Output:** `pc` (current address).

Inside: one 32-bit register. At every rising clock edge: if reset is on, load 0; otherwise load `pc_next`.

Verilog used: `always @(posedge clk)`, `if/else`, `<=`, `output reg`.

```
// ============================================================
// program_counter.v
// Holds the address of the instruction we are executing now.
// On every rising clock edge it loads the "next PC" value.
// If reset = 1, it goes back to address 0 (start of program).
// ============================================================
module program_counter (
    input             clk,      // clock signal
    input             reset,    // 1 = restart from address 0
    input      [31:0] pc_next,  // address of the next instruction
    output reg [31:0] pc        // address of the current instruction
);

    always @(posedge clk) begin
        if (reset)
            pc <= 32'd0;        // start of program
        else
            pc <= pc_next;      // move to the next instruction
    end

endmodule
```

Block by block:

- `output reg [31:0] pc` — `pc` is an output, and it's a `reg` because it's assigned inside an `always` block. It is a real 32-bit register here because the block is clocked.
- `always @(posedge clk)` — only act on the rising edge of the clock.
- `if (reset) pc <= 32'd0;` — reset sends us back to the start of the program.
- `else pc <= pc_next;` — otherwise move to whatever the next-PC MUX decided.

This is a **synchronous reset**: reset is only noticed at a clock edge. Simple and common.

5.4 Module: `instruction_memory`

Purpose: hold the program and return the instruction at a given address.

Input: `addr` (from the PC). **Output:** `instr` (32-bit instruction).

Inside: an array of 64 words. The program is written into it at the start of simulation.

Verilog used: array, `assign`, bit selection `addr[7:2]`, `initial`, `for`.

```
// ============================================================
// instruction_memory.v
```

```
// Stores the program (machine code). Give it an address,
// it gives back the 32-bit instruction stored there.
// It is read-only: the processor never writes to it.
// ============================================================
module instruction_memory (
    input  [31:0] addr,         // byte address (comes from the PC)
    output [31:0] instr         // instruction stored at that address
);

    reg [31:0] mem [0:63];      // 64 slots, each 32 bits wide

    // Each instruction is 4 bytes, so slot number = addr / 4.
    // addr[7:2] means "bits 7 down to 2" = addr divided by 4.
    assign instr = mem[addr[7:2]];

    integer i;
    initial begin
        // Fill all slots with 0 first (0 = empty)
        for (i = 0; i < 64; i = i + 1)
            mem[i] = 32'h00000000;

        // ---------- Our test program ----------
        //   slot      machine code       PC   assembly
        mem[0]  = 32'h00c00093;  //  0  addi x1, x0, 12
        mem[1]  = 32'h00a00113;  //  4  addi x2, x0, 10
        mem[2]  = 32'h002081b3;  //  8  add  x3, x1, x2   -> 22
        mem[3]  = 32'h40208233;  // 12  sub  x4, x1, x2   -> 2
        mem[4]  = 32'h0020f2b3;  // 16  and  x5, x1, x2   -> 8
        mem[5]  = 32'h0020e333;  // 20  or   x6, x1, x2   -> 14
        mem[6]  = 32'h00302023;  // 24  sw   x3, 0(x0)    mem[0] = 22
        mem[7]  = 32'h00002383;  // 28  lw   x7, 0(x0)    x7 = 22
        mem[8]  = 32'h00718463;  // 32  beq  x3, x7, 8    equal -> jump to 40
        mem[9]  = 32'h00100413;  // 36  addi x8, x0, 1    (SKIPPED by beq)
        mem[10] = 32'h06300493;  // 40  addi x9, x0, 99
        mem[11] = 32'h00000063;  // 44  beq  x0, x0, 0    loop forever (stop)
    end

endmodule
```

Block by block:

- `reg [31:0] mem [0:63];` — 64 slots of 32 bits each = room for 64 instructions.
- `assign instr = mem[addr[7:2]];` — addresses count **bytes**, but each slot is 4 bytes. Address 0 → slot 0, address 4 → slot 1, address 8 → slot 2. Dividing by 4 is the same as dropping the lowest 2 bits, so we use bits 7..2. (7..2 = 6 bits = 64 slots.)
- The read is **instant** (no clock) — as soon as the PC changes, the new instruction appears.
- `initial begin ... end` — runs once: first fill everything with 0, then load our 12 program instructions. Each line has a comment showing its address and the assembly it represents.

5.5 Module: `register_file`

Purpose: the 32 registers x0–x31.

Inputs: `clk`, `reg_write` (enable), `rs1`, `rs2` (which registers to read), `rd` (which to write), `write_data` (value to write).
Outputs: `read_data1`, `read_data2`.

Inside: an array of 32 registers. Two read ports that work instantly, one write port that works on the clock edge. Register 0 is forced to read as zero.

Verilog used: array, `initial`/`for`, `assign` with `? :`, `always @(posedge clk)`, `&&`.

```
// ============================================================
// register_file.v
// 32 registers (x0 ... x31), each 32 bits.
// - Two READ ports  : read rs1 and rs2 at the same time (instant)
// - One WRITE port  : write rd on the rising clock edge
// - x0 is always 0 in RISC-V
// ============================================================
module register_file (
    input         clk,
    input         reg_write,    // 1 = write the result into rd
    input  [4:0]  rs1,          // number of 1st source register
    input  [4:0]  rs2,          // number of 2nd source register
    input  [4:0]  rd,           // number of destination register
    input  [31:0] write_data,   // value to write into rd
    output [31:0] read_data1,   // value of register rs1
    output [31:0] read_data2    // value of register rs2
);

    reg [31:0] regs [0:31];     // the 32 registers

    // Start every register at 0
    integer i;
```

```
    initial begin
        for (i = 0; i < 32; i = i + 1)
            regs[i] = 32'd0;
    end

    // Reading: happens immediately (combinational).
    // If someone asks for x0, always give 0.
    assign read_data1 = (rs1 == 5'd0) ? 32'd0 : regs[rs1];
    assign read_data2 = (rs2 == 5'd0) ? 32'd0 : regs[rs2];

    // Writing: happens only on the rising clock edge.
    // Never write into x0.
    always @(posedge clk) begin
        if (reg_write && rd != 5'd0)
            regs[rd] <= write_data;
    end

endmodule
```

Block by block:

- `reg [31:0] regs [0:31];` — the 32 registers.
- The `initial` loop sets them all to 0 so the simulation starts clean.
- `assign read_data1 = (rs1 == 5'd0) ? 32'd0 : regs[rs1];` — reading is combinational. If the register number is 0, output 0 (RISC-V rule), otherwise output the stored value. Same for port 2.
- `always @(posedge clk) if (reg_write && rd != 5'd0) regs[rd] <= write_data;` — writing only happens at the clock edge, only when control says so, and never to x0.

Why read instantly but write on the clock? We must read the inputs during the cycle to calculate the answer, then save the answer exactly once, at the end of the cycle.

5.6 Module: `alu`

Purpose: do the arithmetic/logic.

Inputs: `a`, `b` (32-bit numbers), `alu_ctrl` (3-bit operation code). **Outputs:** `result` (32-bit), `zero` (1 if result is 0).

Inside: a `case` that picks one of four operations. A separate line makes the zero flag.

Verilog used: `always @(*)`, `case`, `+ - & |`, `assign`, `==`.

```
// ============================================================
// alu.v  (Arithmetic Logic Unit)
// Does the actual calculation. The control unit tells it
// WHICH calculation to do using a 3-bit code (alu_ctrl).
//
//   alu_ctrl | operation
//   ---------+-----------
//     000    |  a + b
//     001    |  a - b
//     010    |  a & b
//     011    |  a | b
//
// zero = 1 when the result is 0 (used by BEQ to test a == b)
// ============================================================
module alu (
    input      [31:0] a,          // first number
    input      [31:0] b,          // second number
    input      [2:0]  alu_ctrl,   // which operation to do
    output reg [31:0] result,     // answer
    output            zero        // 1 if result == 0
);

    always @(*) begin
        case (alu_ctrl)
            3'b000:  result = a + b;   // ADD  (also used by ADDI, LW, SW)
            3'b001:  result = a - b;   // SUB  (also used by BEQ)
            3'b010:  result = a & b;   // AND
            3'b011:  result = a | b;   // OR
            default: result = 32'd0;
        endcase
    end

    assign zero = (result == 32'd0);

endmodule
```

Block by block:

- `always @(*)` — whenever `a`, `b`, or `alu_ctrl` changes, recompute `result` immediately.
- `case (alu_ctrl)` — choose the operation. `&` and `|` work **bit by bit** on all 32 bits.

- `default: result = 32'd0;` — safety: an unknown code gives 0 (and avoids accidentally creating memory in the hardware).
- `assign zero = (result == 32'd0);` — 1 if the answer is zero. Only BEQ cares about this.

5.7 Module: `control_unit`

Purpose: decode the instruction and produce the control signals (the table in Part 4.4).

Inputs: `opcode` (instr[6:0]), `funct3` (instr[14:12]), `funct7_5` (instr[30]). **Outputs:** `reg_write`, `alu_src`, `mem_write`, `mem_to_reg`, `branch`, `alu_ctrl`.

Inside: first set everything to "do nothing", then use a `case` on the opcode to switch on the signals each instruction needs. For R-type, a second `case` on `funct3` chooses the ALU operation.

Verilog used: `always @(*)`, nested `case`, `? :`, default values.

```
// ============================================================
// control_unit.v
// The "brain". Looks at the instruction and decides what every
// other part of the processor should do for this instruction.
//
// Inputs come straight from the instruction bits:
//   opcode (bits 6:0)  -> what TYPE of instruction
//   funct3 (bits 14:12)-> which operation inside that type
//   funct7_5 (bit 30)  -> tells ADD (0) apart from SUB (1)
// ============================================================
module control_unit (
    input      [6:0] opcode,
    input      [2:0] funct3,
    input            funct7_5,
    output reg       reg_write,   // 1 = write a result into rd
    output reg       alu_src,     // 0 = ALU uses rs2, 1 = ALU uses immediate
    output reg       mem_write,   // 1 = store into data memory (SW)
    output reg       mem_to_reg,  // 0 = write ALU result, 1 = write memory data (LW)
    output reg       branch,      // 1 = this is a BEQ instruction
    output reg [2:0] alu_ctrl     // which operation the ALU should do
);

    always @(*) begin
        // Default values: "do nothing". Every case below
        // only changes the signals it needs.
        reg_write  = 0;
        alu_src    = 0;
        mem_write  = 0;
        mem_to_reg = 0;
        branch     = 0;
        alu_ctrl   = 3'b000;

        case (opcode)

            7'b0110011: begin              // R-type: ADD, SUB, AND, OR
                reg_write = 1;
                alu_src   = 0;             // second operand is rs2
                case (funct3)
                    3'b000: alu_ctrl = (funct7_5 == 1) ? 3'b001 : 3'b000; // SUB : ADD
                    3'b111: alu_ctrl = 3'b010;  // AND
                    3'b110: alu_ctrl = 3'b011;  // OR
                    default: alu_ctrl = 3'b000;
                endcase
            end

            7'b0010011: begin              // ADDI
                reg_write = 1;
                alu_src   = 1;             // second operand is the immediate
                alu_ctrl  = 3'b000;        // add
            end

            7'b0000011: begin              // LW
                reg_write  = 1;
                alu_src    = 1;            // address = rs1 + immediate
                mem_to_reg = 1;            // write the value read from memory
                alu_ctrl   = 3'b000;       // add
            end

            7'b0100011: begin              // SW
                alu_src   = 1;             // address = rs1 + immediate
                mem_write = 1;             // write rs2 into memory
                alu_ctrl  = 3'b000;        // add
            end

            7'b1100011: begin              // BEQ
                branch   = 1;
                alu_src  = 0;              // compare rs1 with rs2
                alu_ctrl = 3'b001;         // subtract: zero = 1 if equal
            end
```

```
        endcase
    end

endmodule
```

Block by block:

- **Default values at the top.** This is a good beginner habit: every output gets a value in every situation. If we forgot one, Verilog would create unwanted memory (called a “latch”). It also means each instruction’s block only lists the signals that are **on**.
- **R-type block** (`0110011`): write a register, use rs2. Then look at funct3: `000` → ADD or SUB (bit 30 decides), `111` → AND, `110` → OR.
- **ADDI block** (`0010011`): write a register, use the immediate, add.
- **LW block** (`0000011`): write a register, use the immediate (address = rs1 + offset), add, and take the value from memory.
- **SW block** (`0100011`): use the immediate (address = rs1 + offset), add, write memory. `reg_write` stays 0.
- **BEQ block** (`1100011`): compare two registers by subtracting, mark as branch. `reg_write` stays 0.

Design choice: textbooks often split this into a “main control” + an “ALU control” module. We combined them into one so there’s only one place to look. Easier to explain.

5.8 Module: `imm_gen`

Purpose: build the 32-bit immediate from the instruction bits.

Input: `instr`. **Output:** `imm`.

Inside: look at the opcode to know the format, then glue together the right bits and sign-extend.

Verilog used: `always @(*)`, `case` with two values on one line, concatenation `{ }`, replication `{N{ }}`.

```
// ============================================================
// imm_gen.v  (Immediate Generator)
// Some instructions carry a constant number inside them
// (e.g. the 12 in "addi x1, x0, 12"). Different instruction
// types store this number in different bit positions.
// This module collects those bits and makes a 32-bit number.
//
// {20{instr[31]}} means "copy bit 31 twenty times". This is
// called SIGN EXTENSION: it keeps negative numbers negative.
// ============================================================
module imm_gen (
    input      [31:0] instr,
    output reg [31:0] imm
);

    always @(*) begin
        case (instr[6:0])   // look at the opcode

            // I-type (ADDI, LW): immediate is in bits 31:20
            7'b0010011,
            7'b0000011: imm = {{20{instr[31]}}, instr[31:20]};

            // S-type (SW): immediate is split into bits 31:25 and 11:7
            7'b0100011: imm = {{20{instr[31]}}, instr[31:25], instr[11:7]};

            // B-type (BEQ): bits are scattered; last bit is always 0
            // because branch targets are always even addresses
            7'b1100011: imm = {{19{instr[31]}}, instr[31], instr[7],
                               instr[30:25], instr[11:8], 1'b0};

            default:    imm = 32'd0;   // R-type has no immediate
        endcase
    end

endmodule
```

Block by block:

- **I-type** (ADDI, LW): the 12-bit constant is in bits 31..20. `{ {20{instr[31]}}, instr[31:20] }` = 20 copies of the sign bit + the 12 bits = 32 bits.
- **S-type** (SW): the constant is in two pieces: bits 31..25 (upper 7) and 11..7 (lower 5). Glue them: `instr[31:25]`, `instr[11:7]`.
- **B-type** (BEQ): bits are shuffled by the RISC-V designers. We put them back in order: bit 12 (`instr[31]`), bit 11 (`instr[7]`), bits 10..5 (`instr[30:25]`), bits 4..1 (`instr[11:8]`) and then a `0` for bit 0. Branch offsets are always even, so bit 0 is never stored.

- **R-type** has no immediate, so default = 0.

What is sign extension? In 12 bits, `111111111111` means −1. To keep it −1 in 32 bits, we must fill the top 20 bits with 1s, not 0s. Copying the top bit (the "sign bit") does exactly that. For positive numbers the sign bit is 0, so we fill with 0s.

5.9 Module: `data_memory`

Purpose: RAM for LW / SW.

Inputs: `clk`, `mem_write`, `addr` (from ALU), `write_data` (from rs2). **Output:** `read_data`.

Inside: 64 words. Read is instant; write happens on the clock edge if `mem_write = 1`.

```
// ============================================================
// data_memory.v
// The processor's "RAM" for variables.
// - LW reads from it (instantly, combinational)
// - SW writes into it (on the rising clock edge)
// ============================================================
module data_memory (
    input         clk,
    input         mem_write,    // 1 = store write_data at addr
    input  [31:0] addr,         // byte address (from the ALU)
    input  [31:0] write_data,   // value to store (from rs2)
    output [31:0] read_data     // value stored at addr
);

    reg [31:0] mem [0:63];      // 64 words

    integer i;
    initial begin
        for (i = 0; i < 64; i = i + 1)
            mem[i] = 32'd0;
    end

    // word number = addr / 4  (same idea as instruction memory)
    assign read_data = mem[addr[7:2]];

    always @(posedge clk) begin
        if (mem_write)
            mem[addr[7:2]] <= write_data;
    end

endmodule
```

Block by block: exactly like the register file but with one read port and an address from the ALU. `addr[7:2]` again converts a byte address into a word number. Note that the memory *always* outputs something; it's the `mem_to_reg` MUX that decides whether that value is used.

5.10 Module: `riscv_cpu` (top level)

Purpose: connect all the boxes with wires, and add the three MUXes and two adders. This file **is** the datapath diagram, written in text.

Inputs: `clk`, `reset`. That's all — the program is inside instruction memory.

```
// ============================================================
// riscv_cpu.v  (TOP MODULE)
// Connects all the parts with wires to make the full processor.
// The three multiplexers (MUX) are written here directly using
// the "? :" operator, so we don't need a separate mux module.
// ============================================================
module riscv_cpu (
    input clk,
    input reset
);

    // ---------------- Wires (connections) ----------------
    wire [31:0] pc, pc_next, pc_plus4, pc_branch;
    wire [31:0] instr;
    wire [31:0] read_data1, read_data2, imm;
    wire [31:0] alu_b, alu_result, mem_data, write_back;
    wire        zero;

    // control signals
    wire       reg_write, alu_src, mem_write, mem_to_reg, branch;
    wire [2:0] alu_ctrl;

    // ---------------- 1. FETCH ----------------
    program_counter pc_reg (
        .clk(clk), .reset(reset), .pc_next(pc_next), .pc(pc)
    );
```

```
    instruction_memory imem (
        .addr(pc), .instr(instr)
    );

    // ---------------- 2. DECODE ----------------
    // Split the instruction into its fields and send them out.
    control_unit cu (
        .opcode(instr[6:0]),
        .funct3(instr[14:12]),
        .funct7_5(instr[30]),
        .reg_write(reg_write),
        .alu_src(alu_src),
        .mem_write(mem_write),
        .mem_to_reg(mem_to_reg),
        .branch(branch),
        .alu_ctrl(alu_ctrl)
    );

    register_file rf (
        .clk(clk),
        .reg_write(reg_write),
        .rs1(instr[19:15]),
        .rs2(instr[24:20]),
        .rd(instr[11:7]),
        .write_data(write_back),
        .read_data1(read_data1),
        .read_data2(read_data2)
    );

    imm_gen ig (
        .instr(instr), .imm(imm)
    );

    // ---------------- 3. EXECUTE ----------------
    // MUX 1: ALU's second input = rs2 value OR immediate
    assign alu_b = (alu_src == 1) ? imm : read_data2;

    alu alu_unit (
        .a(read_data1), .b(alu_b), .alu_ctrl(alu_ctrl),
        .result(alu_result), .zero(zero)
    );

    // ---------------- 4. MEMORY ----------------
    data_memory dmem (
        .clk(clk),
        .mem_write(mem_write),
        .addr(alu_result),
        .write_data(read_data2),
        .read_data(mem_data)
    );

    // ---------------- 5. WRITE BACK ----------------
    // MUX 2: value written into rd = ALU result OR memory data
    assign write_back = (mem_to_reg == 1) ? mem_data : alu_result;

    // ---------------- 6. NEXT PC ----------------
    assign pc_plus4  = pc + 32'd4;        // normal: next instruction
    assign pc_branch = pc + imm;          // BEQ target

    // MUX 3: jump only if it is a BEQ AND the two registers were equal
    assign pc_next = (branch && zero) ? pc_branch : pc_plus4;

endmodule
```

Block by block, matching the five steps:

1. **Fetch** — the PC register drives the instruction memory's address.
2. **Decode** — the instruction is cut into fields with bit-slices: `instr[6:0]` to control, `instr[19:15]` / `[24:20]` / `[11:7]` to the register file, the whole instruction to the immediate generator.
3. **Execute** — MUX 1 (`alu_b`) chooses rs2 or the immediate. The ALU computes.
4. **Memory** — the ALU result is used as the address; rs2's value is the data for SW.
5. **Write back** — MUX 2 (`write_back`) chooses memory data (LW) or ALU result (everything else). This wire goes back into the register file's `write_data` pin — notice the loop in the diagram.
6. **Next PC** — two adders compute `pc + 4` and `pc + imm`. MUX 3 picks the branch target only if `branch && zero` (it's a BEQ *and* the registers were equal).

Every line here is either a wire declaration, a module placement, or an `assign`. There is no "order of execution" — it's all one connected circuit.

Part 6 — Testbench

6.1 Background concepts

What is a clock? A signal that keeps flipping 0 → 1 → 0 → 1 at a fixed speed. The moment it goes from 0 to 1 is called the **rising edge** (`posedge`). It works like a metronome or a school bell: "everyone, save your work NOW."

Why does the processor need a clock? Without it, nobody would know *when* a result is finished and safe to save. The combinational parts (ALU, muxes) need a little time to settle; the clock gives them that time and then, at the edge, the PC, registers and memory all update together. One clock edge = one instruction completed.

What is reset? A "start from the beginning" button. When `reset = 1`, the PC is set to 0. We hold reset for one cycle at the start so the processor begins at the first instruction, not at a random address.

What is a testbench? A Verilog module that is **not** part of the processor. It's a pretend lab: it creates the clock, presses reset, lets the CPU run, and checks the results. It has no inputs or outputs of its own.

What does `#10` mean? "Wait 10 time units" in the simulation. It doesn't slow your computer down; it moves the simulated clock forward. We imagine 1 unit = 1 ns.

What does `$display` do? Prints text to the terminal, like `printf`. `%d` prints a number in decimal, `%h` in hex, `%0d` without extra spaces.

`$dumpfile` / `$dumpvars` — record every signal into a file (`cpu.vcd`) so you can see waveforms in GTKWave.

`$finish` — stop the simulation.

`cpu.rf.regs[3]` — a *hierarchical name*: "inside the instance `cpu`, inside the instance `rf`, the array `regs`, item 3." It lets the testbench peek inside the processor without adding extra output pins. (Simulation-only trick.)

6.2 The code

```
// ============================================================
// testbench.v
// NOT part of the processor. It is a "virtual lab bench":
// it gives the CPU a clock, presses reset, lets it run,
// and then checks whether the answers are correct.
// ============================================================
// Note: "#10" means "wait 10 time units". We imagine 1 unit = 1 ns.

module testbench;

    reg clk;
    reg reset;
    integer errors;

    // Put one processor on the bench
    riscv_cpu cpu (
        .clk(clk),
        .reset(reset)
    );

    // Clock: flip every 5 ns -> one full cycle every 10 ns
    initial clk = 0;
    always #5 clk = ~clk;

    // Print what happens on every clock cycle
    always @(posedge clk) begin
        if (!reset)
            $display("PC=%2d  instr=%h  x1=%0d x2=%0d x3=%0d x4=%0d x5=%0d x6=%0d x7=%0d x8=%0d x9=%0d  mem[0]=%0d",
                     cpu.pc, cpu.instr,
                     cpu.rf.regs[1], cpu.rf.regs[2], cpu.rf.regs[3],
                     cpu.rf.regs[4], cpu.rf.regs[5], cpu.rf.regs[6],
                     cpu.rf.regs[7], cpu.rf.regs[8], cpu.rf.regs[9],
                     cpu.dmem.mem[0]);
    end

    // Small helper: compare a value with what we expect
    task check;
        input [8*8-1:0] name;   // up to 8 characters of text
        input [31:0] got;
        input [31:0] expected;
        begin
            if (got == expected)
                $display("  PASS  %0s = %0d", name, got);
            else begin
                $display("  FAIL  %0s = %0d (expected %0d)", name, got, expected);
                errors = errors + 1;
            end
        end
    endtask
```

```
    initial begin
        // Save all signals so we can view waveforms in GTKWave
        $dumpfile("cpu.vcd");
        $dumpvars(0, testbench);

        errors = 0;
        reset  = 1;          // hold reset for the first cycle
        #10;
        reset  = 0;          // let the processor run

        #150;                // 15 cycles: enough for 12 instructions

        $display("");
        $display("========== FINAL CHECK ==========");
        check("x1",      cpu.rf.regs[1], 12);
        check("x2",      cpu.rf.regs[2], 10);
        check("x3",      cpu.rf.regs[3], 22);   // ADD
        check("x4",      cpu.rf.regs[4], 2);    // SUB
        check("x5",      cpu.rf.regs[5], 8);    // AND
        check("x6",      cpu.rf.regs[6], 14);   // OR
        check("mem[0]", cpu.dmem.mem[0], 22);  // SW
        check("x7",      cpu.rf.regs[7], 22);   // LW
        check("x8",      cpu.rf.regs[8], 0);    // skipped by BEQ
        check("x9",      cpu.rf.regs[9], 99);   // BEQ target ran
        check("PC",      cpu.pc, 44);           // stuck in final loop

        if (errors == 0)
            $display(">>> ALL TESTS PASSED <<<");
        else
            $display(">>> %0d TEST(S) FAILED <<<", errors);

        $finish;
    end

endmodule
```

6.3 Block by block

- `reg clk; reg reset;` — the testbench drives these, so they are `reg`.
- `riscv_cpu cpu (.clk(clk), .reset(reset));` — place one processor and connect it.
- `initial clk = 0; always #5 clk = ~clk;` — start at 0, flip every 5 units. So one full clock period is 10 units.
- The `always @(posedge clk)` print block — at every rising edge, print the PC, the instruction, and registers x1–x9 and memory word 0. **Each line shows the instruction being executed in that cycle, and the register values *before* its result is saved.** The saved result appears on the next line.
- `task check` — a small helper (like a function) that compares a value with the expected value and prints PASS or FAIL. It counts failures in `errors`.
- The main `initial` block:
  1. start recording waveforms;
  2. hold `reset = 1` for 10 units (one clock cycle), then release it;
  3. wait 150 units = 15 cycles (our program needs 11; the extra cycles just loop at the end);
  4. check every register and the memory word against the hand-calculated answers;
  5. print “ALL TESTS PASSED” or how many failed; stop.

6.4 How do I know my processor is working?

Three ways, from easiest to most detailed:

1. **The final check:** the testbench prints `>>> ALL TESTS PASSED <<<`. Each check compares against an answer we calculated by hand.
2. **The trace:** the per-cycle lines show the PC going 0, 4, 8, … 32, then **40** (skipping 36 because BEQ jumped), then staying at 44 forever. Registers fill up one by one with the expected values.
3. **The waveform:** in GTKWave you can see each signal change at each clock edge (Part 8.5).

Part 7 — Example Program

7.1 The program

It uses **all 8 instructions**, with small numbers so you can do everything in your head.

Address (PC)	Assembly	Machine code (hex)	What it does
0	`addi x1, x0, 12`	`00c00093`	x1 = 12
4	`addi x2, x0, 10`	`00a00113`	x2 = 10
8	`add x3, x1, x2`	`002081b3`	x3 = 12 + 10 = 22
12	`sub x4, x1, x2`	`40208233`	x4 = 12 − 10 = 2
16	`and x5, x1, x2`	`0020f2b3`	x5 = 12 AND 10 = 8
20	`or x6, x1, x2`	`0020e333`	x6 = 12 OR 10 = 14
24	`sw x3, 0(x0)`	`00302023`	Memory[0] = 22
28	`lw x7, 0(x0)`	`00002383`	x7 = Memory[0] = 22
32	`beq x3, x7, 8`	`00718463`	22 == 22 → jump to 32 + 8 = 40
36	`addi x8, x0, 1`	`00100413`	**skipped** (proves BEQ jumped)
40	`addi x9, x0, 99`	`06300493`	x9 = 99
44	`beq x0, x0, 0`	`00000063`	0 == 0 → jump to 44 + 0 = 44 (stop here forever)

Why 12 and 10? In binary 12 = `1100` and 10 = `1010`. They are different in a useful way, so AND and OR give interesting answers:

```
  1100   (12)          1100   (12)
& 1010   (10)        | 1010   (10)
------               ------
  1000   = 8           1110   = 14
```

Why the last instruction? A real program would return to an operating system. We have none, so we make the processor jump to itself forever. This is a common way to “halt” a simple CPU.

Why the instruction at 36? It is a trap. If BEQ works, it is skipped and x8 stays 0. If BEQ is broken, x8 becomes 1 and the test fails.

7.2 Step-by-step execution

Cycle	PC	Instruction	ALU operation	Memory	Register written	Next PC
1	0	addi x1, x0, 12	0 + 12 = 12	—	x1 = 12	4
2	4	addi x2, x0, 10	0 + 10 = 10	—	x2 = 10	8
3	8	add x3, x1, x2	12 + 10 = 22	—	x3 = 22	12
4	12	sub x4, x1, x2	12 − 10 = 2	—	x4 = 2	16
5	16	and x5, x1, x2	12 & 10 = 8	—	x5 = 8	20
6	20	or x6, x1, x2	12 | 10 = 14	—	x6 = 14	24
7	24	sw x3, 0(x0)	0 + 0 = 0 (address)	**write** Mem[0] = 22	—	28
8	28	lw x7, 0(x0)	0 + 0 = 0 (address)	**read** Mem[0] → 22	x7 = 22	32
9	32	beq x3, x7, 8	22 − 22 = 0 → zero = 1	—	—	**40** (jump)
10	40	addi x9, x0, 99	0 + 99 = 99	—	x9 = 99	44
11+	44	beq x0, x0, 0	0 − 0 = 0 → zero = 1	—	—	44 (loop)

7.3 Final state

Register / memory	Final value	Proves that … works
x1	12	ADDI
x2	10	ADDI
x3	22	ADD
x4	2	SUB
x5	8	AND
x6	14	OR
Mem[0]	22	SW
x7	22	LW
x8	0	BEQ (instruction was skipped)
x9	99	BEQ (landed at the right target)
PC	44	Halt loop

Part 8 — Simulation Instructions

We use two free tools:

- **Icarus Verilog** (`iverilog`) — compiles and runs Verilog.
- **GTKWave** — shows waveforms (graphs of signals over time).

8.1 Install

Windows: download the "Icarus Verilog for Windows" installer from **bleyer.org/icarus** (the latest version is fine). Run the installer. **Tick the options** to install GTKWave too and to *add the executables to PATH*. Then open a new Command Prompt and type `iverilog -V` — if it prints a version, it worked.

Ubuntu / Linux:

```
sudo apt install iverilog gtkwave
```

macOS (with Homebrew):

```
brew install icarus-verilog
```

For waveforms on Mac, install GTKWave or the free "Surfer" waveform viewer.

No install possible? Use **EDA Playground** (edaplayground.com) in the browser: choose "Icarus Verilog" as the simulator, paste `testbench.v` in the left pane and all the other files in the right pane (you can add files with the + button), tick "Open EPWave after run", and press Run.

8.2 Create the files

Make a folder, e.g. `riscv_project`, and put these 9 files in it (exact names, all lowercase, ending in `.v`):

```
riscv_project/
    program_counter.v
    instruction_memory.v
    register_file.v
    alu.v
    control_unit.v
    imm_gen.v
    data_memory.v
    riscv_cpu.v
    testbench.v
```

(The ready-made files are provided with this guide — just copy the folder.)

8.3 Compile

Open a terminal / Command Prompt **inside that folder** (`cd riscv_project`) and run:

```
iverilog -o cpu_sim program_counter.v instruction_memory.v register_file.v alu.v control_unit.v imm_gen.v data_memory.v
riscv_cpu.v testbench.v
```

- `iverilog` = the compiler
- `-o cpu_sim` = name the output file `cpu_sim`
- then all the `.v` files (order doesn't matter)

If there are no messages, it compiled successfully. (On Linux/Mac you can type `iverilog -o cpu_sim *.v` as a shortcut.)

8.4 Run

```
vvp cpu_sim
```

`vvp` runs the compiled simulation. The output appears in the terminal (see Part 9). It also creates `cpu.vcd`, the waveform file.

8.5 View waveforms

```
gtkwave cpu.vcd
```

In GTKWave:

1. In the top-left panel (“SST”), click `testbench`, then `cpu`.
2. In the panel below, select signals and press **Append** (or drag them to the “Signals” area). Good ones to add: `clk`, `reset`, `pc`, `instr`, `alu_result`, `zero`, `reg_write`, `mem_write`, `branch`, `pc_next`.
3. Press the **zoom to fit** button (magnifying glass with a square).
4. Right-click `pc` → *Data Format* → *Decimal* to see 0, 4, 8, … instead of hex.

What to point out: the PC changes only at rising clock edges; at PC = 32 `zero` and `branch` are both 1, and the next PC is 40 instead of 36.

8.6 Common problems

Problem	Fix
`iverilog: command not found` / not recognised	Not installed or not in PATH. Reinstall and tick “add to PATH”, then open a **new** terminal.
`Unable to open input file`	You’re not in the right folder, or a file name is spelled differently. Use `dir` (Windows) / `ls` (Linux/Mac) to check.
`Unknown module type: alu`	You forgot to include `alu.v` in the compile command.
Output shows `x` values	Something isn’t connected or reset wasn’t applied. Check the wire names in `riscv_cpu.v`.
GTKWave opens but is empty	You need to add signals (step 2 above).

Part 9 — Expected Results

This is the **actual output** from running the project:

```
VCD info: dumpfile cpu.vcd opened for output.
PC= 0  instr=00c00093  x1=0 x2=0 x3=0 x4=0 x5=0 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC= 4  instr=00a00113  x1=12 x2=0 x3=0 x4=0 x5=0 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC= 8  instr=002081b3  x1=12 x2=10 x3=0 x4=0 x5=0 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC=12  instr=40208233  x1=12 x2=10 x3=22 x4=0 x5=0 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC=16  instr=0020f2b3  x1=12 x2=10 x3=22 x4=2 x5=0 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC=20  instr=0020e333  x1=12 x2=10 x3=22 x4=2 x5=8 x6=0 x7=0 x8=0 x9=0  mem[0]=0
PC=24  instr=00302023  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=0 x8=0 x9=0  mem[0]=0
PC=28  instr=00002383  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=0 x8=0 x9=0  mem[0]=22
PC=32  instr=00718463  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=0  mem[0]=22
PC=40  instr=06300493  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=0  mem[0]=22
PC=44  instr=00000063  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=99  mem[0]=22
PC=44  instr=00000063  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=99  mem[0]=22
PC=44  instr=00000063  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=99  mem[0]=22
PC=44  instr=00000063  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=99  mem[0]=22
PC=44  instr=00000063  x1=12 x2=10 x3=22 x4=2 x5=8 x6=14 x7=22 x8=0 x9=99  mem[0]=22

========== FINAL CHECK ==========
  PASS  x1 = 12
  PASS  x2 = 10
  PASS  x3 = 22
  PASS  x4 = 2
  PASS  x5 = 8
  PASS  x6 = 14
  PASS  mem[0] = 22
  PASS  x7 = 22
  PASS  x8 = 0
  PASS  x9 = 99
  PASS  PC = 44
>>> ALL TESTS PASSED <<<
testbench.v:82: $finish called at 160 (1s)
```

How to read it:

- Each `PC=` line is one clock cycle. It shows the instruction *being executed* in that cycle and the register values *just before* it saves its result. That's why `x1` becomes 12 on the line *after* `PC= 0`.
- The PC goes 0, 4, 8 … 28, 32, then **40** — the jump over 36 is the BEQ working.
- `mem[0]` becomes 22 after the SW at PC 24.
- From PC 44 the program loops forever, which is what we wanted.
- All 11 checks pass.

Part 10 — Detailed Explanation: Every Instruction Inside the Processor

For each instruction we follow the same 7 steps. The two clocked actions (save result, update PC) happen **together** at the end of the cycle.

10.1 ADD — `add x3, x1, x2` (x3 = x1 + x2)

1. **PC selects the instruction.** PC = 8 goes to the instruction memory.
2. **Instruction memory returns it:** `002081b3`.
3. **Control unit decodes it.** opcode `0110011` = R-type; funct3 = 000 and bit 30 = 0 → ADD. Signals: `reg_write=1, alu_src=0, mem_write=0, mem_to_reg=0, branch=0, alu_ctrl=000`.
4. **Register file reads** rs1 = x1 (12) and rs2 = x2 (10).
5. **ALU** gets A = 12 and (since `alu_src = 0`) B = 10. Adds: **22**. zero = 0.
6. **Memory:** not used (`mem_write = 0`). **Write back:** `mem_to_reg = 0`, so the ALU result 22 is chosen and, at the clock edge, saved into **x3**.
7. **Next PC:** `branch = 0`, so next PC = 8 + 4 = **12**.

10.2 SUB — `sub x4, x1, x2` (x4 = x1 − x2)

Same path as ADD. The only difference is in step 3: bit 30 of the instruction is **1**, so the control unit sets `alu_ctrl = 001` (subtract). ALU: 12 − 10 = **2** → saved in x4. Next PC = 16.

10.3 AND — `and x5, x1, x2`

Same path as ADD. funct3 = `111` → `alu_ctrl = 010`. ALU: 1100 & 1010 = 1000 = **8** → x5. Next PC = 20.

10.4 OR — `or x6, x1, x2`

Same path as ADD. funct3 = `110` → `alu_ctrl = 011`. ALU: 1100 | 1010 = 1110 = **14** → x6. Next PC = 24.

Key point for the viva: *all four R-type instructions use exactly the same path*. Only the ALU operation changes.

10.5 ADDI — `addi x1, x0, 12` (x1 = x0 + 12)

1. PC = 0 → instruction `00c00093`.
2. Control: opcode `0010011` → `reg_write=1, alu_src=1, alu_ctrl=000`.
3. Register file reads rs1 = x0 → **0**.
4. **Immediate generator** (I-type) takes bits 31..20 = 12, sign-extends → 32-bit 12.
5. **MUX 1:** `alu_src = 1`, so ALU input B = **imm = 12** (not a register!).
6. ALU: 0 + 12 = **12**. Write back MUX picks ALU result → saved in x1.
7. Next PC = 4.

Difference from ADD: the second number comes from the instruction, not from a register. That's the job of MUX 1.

10.6 LW — `lw x7, 0(x0)` (x7 = Memory[x0 + 0])

1. PC = 28 → instruction `00002383`.
2. Control: opcode `0000011` → `reg_write=1, alu_src=1, mem_to_reg=1, alu_ctrl=000`.
3. Register file reads rs1 = x0 → 0.
4. Imm gen (I-type) → offset 0.
5. ALU: 0 + 0 = **0**. This is the **memory address**, not the final answer.
6. **Data memory** reads address 0 → **22** (stored earlier by SW).
7. **MUX 2:** `mem_to_reg = 1` → choose memory data 22 → saved in **x7**.
8. Next PC = 32.

LW uses every part of the processor (instruction memory, register file, imm gen, ALU, data memory, write back). It is the

slowest instruction, and it decides how fast the clock can be.

10.7 SW — `sw x3, 0(x0)` (Memory[x0 + 0] = x3)

1. PC = 24 → instruction `00302023`.
2. Control: opcode `0100011` → `alu_src=1, mem_write=1, reg_write=0, alu_ctrl=000`.
3. Register file reads rs1 = x0 → 0 (base address) and rs2 = x3 → **22** (the data to store).
4. Imm gen (**S-type**: bits 31..25 and 11..7) → offset 0.
5. ALU: 0 + 0 = **0** = address.
6. **Data memory:** `mem_write = 1`, so at the clock edge, Memory[0] = 22.
7. **No register is written** (`reg_write = 0`). Next PC = 28.

Note: rs2's value travels **around** the ALU directly to the memory's `write_data` pin. The ALU is busy computing the address.

10.8 BEQ — `beq x3, x7, 8` (if x3 == x7, PC = PC + 8)

1. PC = 32 → instruction `00718463`.
2. Control: opcode `1100011` → `branch=1, alu_src=0, alu_ctrl=001 (sub), reg_write=0, mem_write=0`.
3. Register file reads x3 = 22 and x7 = 22.
4. ALU: 22 − 22 = **0** → **zero = 1**.
5. Imm gen (**B-type**) → offset **8**. Branch adder: 32 + 8 = **40**.
6. **MUX 3:** `branch AND zero = 1 AND 1 = 1` → next PC = **40** (not 36).
7. No register or memory is written.

If x3 and x7 were different, the ALU result would not be 0, `zero = 0`, and next PC would be 32 + 4 = 36 (branch "not taken").

Why subtract to compare? `a == b` is the same as `a − b == 0`. We already have a subtractor in the ALU, so we reuse it instead of building a separate comparator.

10.9 Summary: which parts each instruction uses

Part	ADD/SUB/AND/OR	ADDI	LW	SW	BEQ
Instruction memory	✔	✔	✔	✔	✔
Register read	rs1, rs2	rs1	rs1	rs1, rs2	rs1, rs2
Imm gen	—	✔ (I)	✔ (I)	✔ (S)	✔ (B)
ALU does	the operation	add	address	address	compare (sub)
Data memory	—	—	read	write	—
Register write	✔	✔	✔	—	—
Next PC	PC+4	PC+4	PC+4	PC+4	PC+4 or PC+imm

Part 11 — Viva Questions and Answers

Answers are written the way a student would say them. Learn the **idea**, not the exact words.

11.1 Basics

Q1. What is RISC-V? An open, free instruction set — a fixed list of commands a processor understands. Anyone can design a RISC-V chip without paying a licence. It's simple and regular, so it's great for learning.

Q2. Why did you choose RISC-V? Three reasons: it's free and open, its instructions are simple and all 32 bits long, and it's used in real industry, so what we learn is real. Also most computer architecture textbooks now use it.

Q3. What does RISC mean? What is the opposite? Reduced Instruction Set Computer: few, simple instructions, each doing one small job. The opposite is CISC (Complex Instruction Set Computer), like x86, where one instruction can do many things and instructions have different lengths.

Q4. What does single-cycle mean? Every instruction is completely finished in one clock cycle — fetch, decode, execute, memory and write back all happen in that one cycle.

Q5. Why is it called single-cycle? Because CPI (cycles per instruction) is exactly 1 for every instruction. At each rising clock edge, one instruction's results are saved and the PC moves to the next one.

Q6. What is the disadvantage of single-cycle? The clock period has to be long enough for the slowest instruction (LW). So even a fast instruction like ADD has to wait for that long period. It wastes time. Pipelining fixes this.

Q7. Which is the slowest instruction in your processor and why? LW, because it goes through every part: instruction memory → register file → ALU → data memory → write back.

Q8. Is this a complete RISC-V processor? No. It implements 8 instructions of the RV32I base set (which has about 40). It's a working subset that shows all the main ideas. It's enough to run small programs with arithmetic, memory and decisions.

Q9. What instructions does your processor support? ADD, SUB, AND, OR (R-type), ADDI and LW (I-type), SW (S-type), and BEQ (B-type).

Q10. Why only these 8? They're the smallest set that covers every type of datapath use: register maths, logic, constants, memory read, memory write, and branching. More instructions would make it bigger without teaching a new concept.

11.2 Components

Q11. What is the role of the PC? It holds the address of the instruction we're executing. Every clock edge it is updated — normally to PC + 4, or to the branch target for a taken BEQ.

Q12. Why does the PC normally increase by 4? Each instruction is 32 bits = 4 bytes, and memory is addressed in bytes. So the next instruction starts 4 bytes later.

Q13. What is a register? A small, very fast storage location inside the processor that holds one value — 32 bits in our case.

Q14. What is the register file? The collection of all 32 registers, x0 to x31, with two read ports and one write port.

Q15. Why two read ports and one write port? Instructions like `add x3, x1, x2` need two input values at the same time, but only produce one result.

Q16. Why is x0 always zero? It's a RISC-V rule. It's useful: `addi x1, x0, 5` loads a constant, `beq x0, x0, label` is an unconditional jump. In our code, reads of x0 always return 0 and writes to x0 are ignored.

Q17. What does the ALU do? It's the calculator. It takes two 32-bit numbers and does add, subtract, AND or OR depending on `alu_ctrl`. It also gives a `zero` flag.

Q18. What is the zero flag used for? For BEQ. The ALU subtracts the two registers; if the result is zero they were equal, so zero = 1 and the branch is taken.

Q19. What is instruction memory? Memory that holds the program. The PC gives the address, and it returns the 32-bit instruction. The processor only reads from it.

Q20. What is data memory? The RAM for data. LW reads from it and SW writes to it. The address comes from the ALU.

Q21. Why are instruction and data memory separate? So that in one cycle we can fetch an instruction *and* read/write data at the same time. With one memory we'd need two cycles for LW/SW. (This separation is called Harvard architecture.)

Q22. What does the control unit do? It looks at the opcode (and funct3/funct7) and generates control signals that tell every other part what to do: whether to write a register, use the immediate, write memory, choose memory data, branch,

and which ALU operation.

Q23. Name the control signals and their meaning. `reg_write` (save to rd), `alu_src` (ALU uses immediate), `mem_write` (write memory), `mem_to_reg` (save memory data instead of ALU result), `branch` (this is BEQ), `alu_ctrl` (ALU operation).

Q24. What is a multiplexer? Why do we need it? A selector switch — several inputs, one output, a select signal chooses which input goes through. We need them because different instructions need data from different places: ALU input B (register or constant), write-back value (ALU or memory), and next PC (PC+4 or branch target).

Q25. How many multiplexers are there? Where? Three: before the ALU (`alu_src`), before write back (`mem_to_reg`), and for the next PC (`branch AND zero`).

Q26. What is an immediate value? A constant number stored inside the instruction itself, like the 12 in `addi x1, x0, 12` or the offset in `lw x7, 0(x0)`.

Q27. What does the immediate generator do? It extracts the immediate bits from the instruction (they're in different positions for I, S and B types), puts them in order, and sign-extends to 32 bits.

Q28. What is sign extension? Making a small signed number bigger without changing its value, by copying the sign bit into the new upper bits. So −1 in 12 bits stays −1 in 32 bits.

Q29. Why is the immediate split into pieces in S and B type? So that rs1, rs2 and rd stay in the same bit positions in every format. The register file can then read registers without waiting to know the instruction type.

Q30. Why does the B-type immediate have a 0 at the end? Branch targets are always at even addresses, so the lowest bit is always 0. RISC-V doesn't waste a bit storing it, which doubles the branch range.

11.3 Instruction working

Q31. What happens during ADD? PC fetches it, control sets reg_write=1 and alu_ctrl=add, register file reads rs1 and rs2, ALU adds, result is written to rd at the clock edge, PC becomes PC+4.

Q32. How does the processor tell ADD and SUB apart? Same opcode and funct3. Bit 30 (from funct7) is 0 for ADD and 1 for SUB.

Q33. What happens during ADDI? Same as ADD, except the ALU's second input comes from the immediate generator instead of rs2 (`alu_src = 1`).

Q34. What happens during LW? ALU adds rs1 + offset to get the address. Data memory reads that address. The write-back MUX picks the memory data (`mem_to_reg = 1`) and saves it into rd.

Q35. What happens during SW? ALU adds rs1 + offset to get the address. The value of rs2 goes to the data memory and is written at the clock edge (`mem_write = 1`). No register is written.

Q36. Why does SW not write a register? It has no destination register — its job is to put a value into memory, not into a register. So `reg_write = 0`.

Q37. What happens during BEQ? Register file reads rs1 and rs2, ALU subtracts them. If the result is zero, `zero = 1`. Meanwhile PC + imm is computed. If `branch = 1` and `zero = 1`, next PC = PC + imm; otherwise PC + 4. Nothing is written.

Q38. Why does the ALU do addition for LW and SW? To calculate the memory address = base register + offset.

Q39. Where does the branch target address come from? A separate adder computes PC + immediate (from the B-type imm generator).

Q40. Why do we need a separate adder for PC + 4 and the branch target? Why not use the ALU? In a single-cycle design, everything happens in the same cycle, and the ALU is already busy (computing a result, an address or a comparison). So we need extra adders.

11.4 Clock, reset, Verilog, testing

Q41. What is the purpose of the clock? It synchronises the processor. At every rising edge the PC, register file and data memory update together. Between edges, the combinational logic calculates.

Q42. What is reset? A signal that puts the processor in a known starting state. In our design it sets PC to 0 so the program starts from the first instruction.

Q43. Why did you use Verilog? It's a hardware description language — it describes circuits, not software steps. It's industry standard, it can be simulated for free (Icarus Verilog), and the same code could be put on an FPGA.

Q44. Difference between `wire` and `reg`? A `wire` is a connection driven by `assign` or a module output. A `reg` gets its value inside an `always` block. A `reg` inside a clocked `always` becomes a real storage element.

Q45. Difference between `always @(*)` and `always @(posedge clk)`? `@(*)` is combinational — it recomputes immediately whenever an input changes (ALU, control). `@(posedge clk)` is sequential — it updates only at the rising clock edge (PC,

register write, memory write).

Q46. Difference between `=` and `<=`? `=` is blocking, used in combinational blocks. `<=` is non-blocking, used in clocked blocks so all registers update together at the edge, like real flip-flops.

Q47. What is a testbench? A Verilog module that tests the design. It creates the clock and reset, runs the processor, and checks results. It is not part of the hardware.

Q48. How did you verify your processor? We wrote a program that uses all 8 instructions, calculated the expected register and memory values by hand, and the testbench automatically compares them. All 11 checks pass. We also checked the waveforms in GTKWave.

Q49. What does `#10` mean? Wait 10 time units in simulation.

Q50. What is `$display`? Prints a message in simulation, like `printf`.

Q51. How does your program stop? The last instruction `beq x0, x0, 0` always branches to itself, so the processor loops forever. The testbench stops the simulation with `$finish`.

11.5 Limitations and future scope

Q52. What are the limitations of your processor? Only 8 instructions; no jumps (JAL/JALR), no other branches, no shifts, no multiply/divide; only word (32-bit) loads/stores; small memories (64 words each); program is hard-coded in instruction memory; no pipelining, so it's slow; no interrupts or exceptions; unsupported instructions aren't detected.

Q53. What would you add to make it more advanced? First, more RV32I instructions (BNE, BLT, JAL, JALR, LUI, shifts, SLT) so it can run real C programs. Then convert it to a 5-stage pipeline with hazard detection and forwarding. Later: caches, interrupts, and putting it on an FPGA board.

Q54. What is pipelining? Splitting the processor into stages (e.g. 5) with registers between them, so that different instructions are in different stages at the same time — like an assembly line. It increases speed but adds problems called hazards.

11.6 Follow-up questions a professor might ask

F1. If ADD is faster than LW, why does ADD take the same time? Because the clock period is fixed and must fit the slowest instruction. That's the main weakness of single-cycle.

F2. What happens if the control unit gets an unknown opcode? All signals stay at their default values (no write, no branch), so the instruction acts like a "do nothing" and the PC just moves to PC + 4.

F3. What happens if you write to x0? Nothing. The register file checks `rd != 0` before writing, and reads of x0 always return 0.

F4. What if the branch offset is negative? The immediate generator sign-extends it, so PC + imm goes backwards. That's how loops work.

F5. Why use `addr[7:2]` in the memories? Addresses are byte addresses; each word is 4 bytes. Dropping the bottom 2 bits = dividing by 4 = word index. 6 bits (7 to 2) give 64 words.

F6. What would you need to add for BNE? Almost nothing: the control unit needs to recognise funct3 = 001, and the branch condition becomes "branch AND NOT zero" for BNE. One extra signal and a small change to the next-PC logic.

F7. What would you need to add for JAL? A path that writes PC + 4 into rd (one more input to the write-back MUX), the J-type immediate in the imm generator, and the next PC = PC + imm always.

F8. Is the register file read combinational or clocked? Why? Reading is combinational (instant) so the values are ready during the same cycle. Writing is clocked so the result is saved exactly once at the end of the cycle.

F9. What is a latch and how did you avoid it? A latch is unwanted memory created when an `always @(*)` block doesn't assign a signal in every case. We avoided it by giving every output a default value at the top of the control unit, and a `default` in every `case`.

F10. Can this run on a real FPGA? Mostly yes — the logic is synthesizable. The `initial` blocks for memory would need to be replaced by memory initialisation files supported by the FPGA tools, and we'd need some way to see outputs (LEDs or a display).

F11. Show me the bits of `addi x1, x0, 12`. `000000001100 00000 000 00001 0010011` → imm = 12, rs1 = 0, funct3 = 000, rd = 1, opcode = ADDI.

F12. What is CPI? Cycles Per Instruction. For a single-cycle processor, CPI = 1.

F13. What is the difference between Harvard and von Neumann architecture? Harvard has separate instruction and data memory (what we used). Von Neumann uses one shared memory for both.

F14. Why are there no `x` (unknown) values in your simulation? Because we initialised all memories and registers to 0 and applied reset to the PC at the start.

Part 12 — Presentation: 10 Slides with Speaking Notes

Keep each slide light. One idea per slide. The datapath slide is the most important.

Slide 1 — Title

On slide: “Design and Implementation of a Single-Cycle RISC-V Processor in Verilog” · team names · roll numbers · guide’s name · college logo. **Say:** “Good morning. Our project is a simple single-cycle processor based on the RISC-V instruction set, written in Verilog and verified by simulation.”

Slide 2 — Problem / Objective

On slide: 3 bullets: understand how a processor executes instructions · design one in Verilog · verify it with simulation. **Say:** “We use processors every day but rarely see what happens inside. Our goal was to build the smallest processor that still shows the full fetch–decode–execute–memory–write back cycle, and prove it works.”

Slide 3 — What is RISC-V?

On slide: “Open, free instruction set” · “simple, fixed 32-bit instructions” · “used in real chips”. Small table: CISC vs RISC. **Say:** “RISC-V is an open instruction set. Unlike x86 or ARM, anyone can use it for free. Its instructions are simple and regular, which makes it ideal for learning.”

Slide 4 — Single-Cycle Architecture

On slide: a clock waveform with one instruction per cycle; the 5 steps in a row: Fetch → Decode → Execute → Memory → Write back. **Say:** “Single-cycle means every instruction completes in one clock cycle. All five steps happen within that cycle, and at the clock edge the results are saved. It’s simple, but the clock must be slow enough for the slowest instruction.”

Slide 5 — Datapath (main slide)

On slide: the datapath diagram from Part 3, big. Colour the three muxes. **Say:** Walk the diagram left to right: “The PC gives the address, instruction memory gives the instruction, the control unit decodes it, the register file reads two registers, the ALU calculates, data memory is used for loads and stores, and the result is written back. The next-PC mux chooses PC + 4 or the branch target.” *(Practise this — it should take about one minute.)*

Slide 6 — Main Components

On slide: a table of the 7 modules with a one-line job for each (PC, instruction memory, register file, ALU, control unit, immediate generator, data memory). **Say:** “Each block is a separate Verilog module. For example, the ALU does add, subtract, AND and OR; the control unit looks at the opcode and generates six control signals.”

Slide 7 — Instruction Set & Execution

On slide: the 8 instructions table, plus the control-signal truth table (Part 4.4). **Say:** “We support eight instructions covering arithmetic, logic, immediate, load, store and branch. The control table shows which signals are on for each. For example, LW uses the immediate, reads memory, and writes a register.”

Slide 8 — Verilog Implementation

On slide: list of the 9 files; one short code snippet — the ALU `case` block or the next-PC line: `assign pc_next = (branch && zero) ? pc_branch : pc_plus4;` **Say:** “Each component is a module. Combinational parts use `always @(*)`; the PC, registers and memory writes use `always @(posedge clk)`. The top module `riscv_cpu` connects everything — it is the datapath written in code.”

Slide 9 — Simulation & Results

On slide: the test program (12 lines), a screenshot of the terminal output with “ALL TESTS PASSED”, and a GTKWave screenshot showing PC jumping from 32 to 40. **Say:** “We wrote a test program using all eight instructions and calculated the expected answers by hand. The testbench checks them automatically — all 11 checks pass. In the waveform you can see the BEQ skipping address 36.”

Slide 10 — Conclusion & Future Scope

On slide: “Working 8-instruction single-cycle RISC-V CPU, verified” · Future: more instructions (JAL, BNE, shifts), 5-stage pipeline, FPGA implementation. **Say:** “We built and verified a working processor and understood every part of it. Next steps would be adding more instructions and converting it into a pipelined design. Thank you — we’re happy to take questions.”

Demo tip: if you can, run `vvp cpu_sim` live. It takes one second and is very convincing.

Part 13 — One-Day Learning Plan

Time	Topic	What to do	You're done when you can …
Hour 1	RISC-V + single-cycle	Read Parts 1 and 2.	Explain what RISC-V and single-cycle mean, and list the 5 steps.
Hour 2	Datapath	Read Part 3. Draw the datapath **from memory** 3 times.	Draw it in under 2 minutes and say what every arrow carries.
Hour 3	Verilog basics	Read Part 5.1–5.2.	Explain `wire` vs `reg`, `assign`, `always @(*)` vs `always @(posedge clk)`.
Hour 4	Individual modules	Read 5.3–5.9 with the code open. Learn the control table (Part 4.4).	Explain any module's inputs, outputs, and what's inside; write the control table from memory.
Break			
Hour 5	Connecting the processor	Read 5.10. Match every line of `riscv_cpu.v` to the diagram.	Point to any wire in the diagram and find it in the code.
Hour 6	Testbench + simulation	Read Parts 6 and 8. Install Icarus, compile, run, open GTKWave.	Run the simulation yourself and see "ALL TESTS PASSED".
Hour 7	Example execution	Read Parts 7, 9 and 10. Fill in the execution table on paper without looking. Then change the program (e.g. `addi x2, x0, 5`) and predict the new results before running.	Explain all 8 instructions using the 7-step pattern.
Hour 8	Viva + presentation	Read Parts 11 and 12. Have a friend ask random viva questions. Practise slide 5 out loud.	Answer the viva questions without reading.

If you only have 2 hours: Part 2.3–2.4, the datapath (Part 3), the control table (Part 4.4), and Part 10.

A good exercise: change one value in the program, predict the result on paper, then simulate. If your prediction matches, you truly understand the processor. (To get new machine code for a changed instruction, change only the immediate field — e.g. `addi x2, x0, 5` is `00500113`.)

Part 14 — Project Report

DESIGN AND IMPLEMENTATION OF A SINGLE-CYCLE RISC-V PROCESSOR USING VERILOG HDL

Semester Project Report · Department of Information Technology

Submitted by: _____ · Roll No.: _____

Under the guidance of: _____

Abstract

This project presents the design, implementation and verification of a simple 32-bit single-cycle processor based on the RISC-V instruction set architecture. The processor supports eight instructions from the RV32I base set — ADD, SUB, AND, OR, ADDI, LW, SW and BEQ — which together cover arithmetic, logical, immediate, memory and conditional-branch operations. The design is written in Verilog HDL as seven functional modules (program counter, instruction memory, register file, ALU, control unit, immediate generator and data memory) connected in a top-level module. It was simulated using Icarus Verilog and GTKWave. A test program exercising every supported instruction was executed, and all eleven automatic checks of register and memory contents passed, confirming correct operation.

Chapter 1 — Introduction

1.1 What is a processor? A processor (CPU) is the part of a computer that executes programs. A program is a sequence of instructions stored in memory. The processor repeatedly fetches an instruction, works out what it means, performs the operation, and moves on to the next instruction.

1.2 What is RISC? RISC (Reduced Instruction Set Computer) is a design philosophy in which the processor supports a small number of simple, fixed-length instructions, each performing one basic operation. Complex tasks are built from sequences of simple instructions. This makes the hardware simpler and easier to make fast. The contrasting approach, CISC (e.g. x86), uses many complex, variable-length instructions.

1.3 What is RISC-V? RISC-V is an open-standard instruction set architecture developed at the University of California, Berkeley. It defines a small integer base set (RV32I for 32-bit) and optional extensions (multiplication, floating point, etc.). All base instructions are 32 bits wide and use a few regular formats.

1.4 Why RISC-V? It is free of licensing restrictions, it is simple and regular (ideal for learning), it is widely used in both education and industry, and its fixed instruction formats make decoding hardware straightforward.

1.5 Objectives. (1) To understand how a processor executes instructions. (2) To design a minimal single-cycle RISC-V datapath and control unit. (3) To implement it in Verilog. (4) To verify it by simulation with a test program.

Chapter 2 — Single-Cycle Processor

2.1 Meaning of single-cycle. In a single-cycle processor, each instruction is executed completely in one clock cycle, giving a CPI (cycles per instruction) of 1. All state elements (PC, register file, data memory) are updated at the rising edge of the clock; all other logic is combinational.

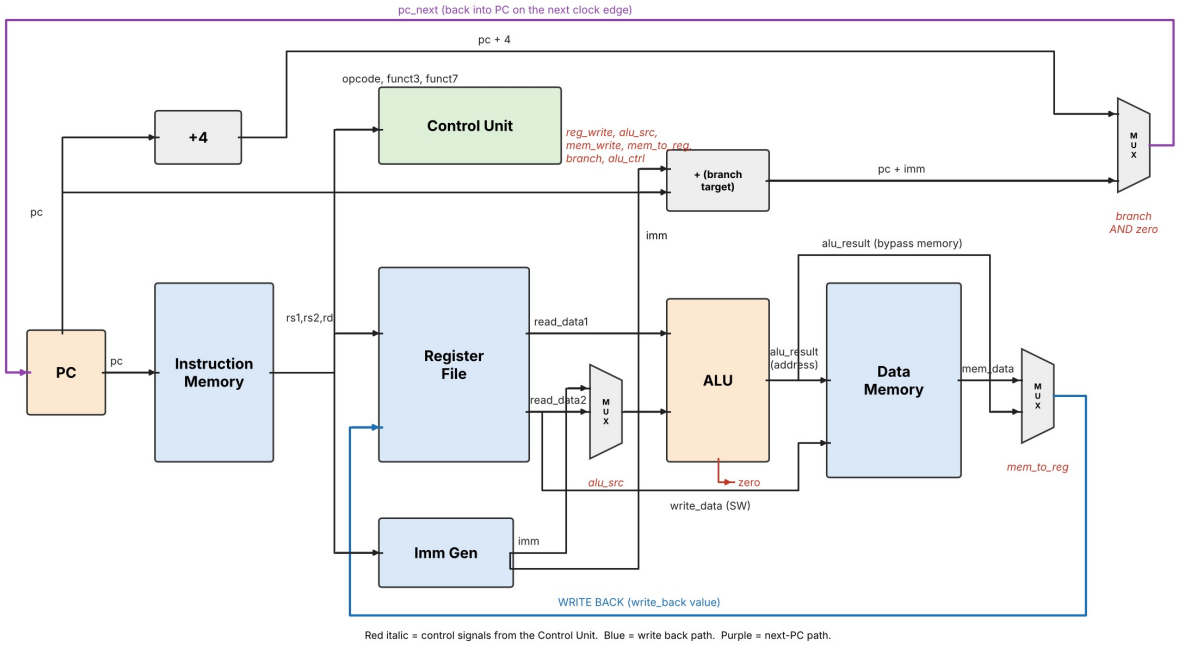
2.2 Basic architecture. The processor consists of state elements (PC, register file, data memory), combinational elements (ALU, control unit, immediate generator, adders, multiplexers) and a read-only instruction memory. Separate instruction and data memories (Harvard organisation) allow an instruction fetch and a data access in the same cycle.

2.3 Instruction execution flow.

Stage	Operation
Fetch	Instruction memory outputs the instruction at address PC.
Decode	Control unit decodes the opcode; register file reads rs1 and rs2; immediate generator produces the immediate.
Execute	ALU operates on rs1 and (rs2 or immediate).
Memory	Data memory is read (LW) or written (SW) at the ALU address.
Write back	The ALU result or memory data is written to rd.
Next PC	PC ← PC + 4, or PC + immediate for a taken branch.

2.4 Advantages and disadvantages. The design is simple, with straightforward control and no hazards. However, the clock period must accommodate the slowest instruction (LW), so faster instructions waste time, and each hardware unit is used only once per cycle.

Chapter 3 — Architecture



Datapath

3.1 Program Counter. A 32-bit register holding the address of the current instruction. On reset it is cleared to 0; otherwise at each rising edge it loads the next-PC value.

3.2 Instruction Memory. A 64-word read-only memory holding the program. It is byte-addressed; the word index is `addr[7:2]`. Reading is combinational.

3.3 Register File. 32 general-purpose 32-bit registers with two combinational read ports and one synchronous write port. Register x0 is hard-wired to zero.

3.4 ALU. Performs ADD, SUB, AND and OR on two 32-bit operands selected by a 3-bit `alu_ctrl` code, and outputs a `zero` flag.

3.5 Control Unit. Decodes the opcode, funct3 and bit 30 of the instruction to produce the control signals `reg_write`, `alu_src`, `mem_write`, `mem_to_reg`, `branch` and `alu_ctrl`.

3.6 Immediate Generator. Extracts and sign-extends the immediate for I-type, S-type and B-type instructions.

3.7 Data Memory. A 64-word memory with a combinational read and synchronous write controlled by `mem_write`.

3.8 Multiplexers. Three 2-to-1 multiplexers select (a) the ALU's second operand (`alu_src`), (b) the write-back value (`mem_to_reg`), and (c) the next PC (`branch AND zero`).

3.9 Branch logic. Two adders compute PC + 4 and PC + immediate. For BEQ the ALU subtracts the operands; if the result is zero and `branch` is asserted, the branch target is selected.

Chapter 4 — Instruction Set

Instruction	Format	Operation	opcode	funct3	funct7[5]
ADD rd, rs1, rs2	R	rd = rs1 + rs2	0110011	000	0
SUB rd, rs1, rs2	R	rd = rs1 − rs2	0110011	000	1
AND rd, rs1, rs2	R	rd = rs1 & rs2	0110011	111	0
OR rd, rs1, rs2	R	rd = rs1 | rs2	0110011	110	0
ADDI rd, rs1, imm	I	rd = rs1 + imm	0010011	000	—
LW rd, imm(rs1)	I	rd = Mem[rs1 + imm]	0000011	010	—
SW rs2, imm(rs1)	S	Mem[rs1 + imm] = rs2	0100011	010	—
BEQ rs1, rs2, imm	B	if rs1 == rs2: PC = PC + imm	1100011	000	—

Control signal table

Instr	reg_write	alu_src	mem_write	mem_to_reg	branch	alu_ctrl
R-type	1	0	0	0	0	000/001/010/011
ADDI	1	1	0	0	0	000
LW	1	1	0	1	0	000
SW	0	1	1	X	0	000
BEQ	0	0	0	X	1	001

Chapter 5 — Verilog Design

The design follows a modular structure; each hardware block is one Verilog module.

Module	Type of logic	Verilog constructs used
program_counter	Sequential	`always @(posedge clk)`, synchronous reset
instruction_memory	Combinational read	array, `assign`, `initial`
register_file	Comb. read, sequential write	array, `assign`, `always @(posedge clk)`
alu	Combinational	`always @(*)`, `case`
control_unit	Combinational	`always @(*)`, nested `case`, default values
imm_gen	Combinational	concatenation, replication (sign extension)
data_memory	Comb. read, sequential write	array, `always @(posedge clk)`
riscv_cpu	Structural (top)	module instantiation, `assign` multiplexers
testbench	Simulation only	`initial`, `#delay`, `$display`, `$dumpvars`, `task`

Key design decisions: (1) combinational logic uses `always @(*)` with blocking assignments, and sequential logic uses `always @(posedge clk)` with non-blocking assignments; (2) all control outputs have default values to avoid inferred latches; (3) main control and ALU control are combined into a single control unit for clarity; (4) multiplexers are expressed with the conditional operator rather than separate modules; (5) no parameters are used, so all widths are explicit. The full source code with explanations is given in Part 5 of this document.

Chapter 6 — Datapath

The instruction flows from the PC to the instruction memory. Its fields drive the control unit (opcode, funct3, funct7), the register file (rs1, rs2, rd) and the immediate generator. Register value rs1 is the ALU's first operand; the ALU's second operand is either rs2 or the immediate. The ALU result is used either as the value to write back (R-type, ADDI) or as the data memory address (LW, SW). For SW, rs2 is routed to the memory's write-data port. The write-back multiplexer selects the ALU result or the memory output and returns it to the register file. In parallel, PC + 4 and PC + immediate are computed, and the next-PC multiplexer chooses between them based on `branch AND zero`. At the rising clock edge, the PC, the

destination register and (for SW) the memory are updated simultaneously.

Chapter 7 — Example Execution

PC	Instruction	ALU	Memory	Result
0	addi x1, x0, 12	0 + 12	—	x1 = 12
4	addi x2, x0, 10	0 + 10	—	x2 = 10
8	add x3, x1, x2	12 + 10	—	x3 = 22
12	sub x4, x1, x2	12 − 10	—	x4 = 2
16	and x5, x1, x2	12 & 10	—	x5 = 8
20	or x6, x1, x2	12 | 10	—	x6 = 14
24	sw x3, 0(x0)	0 + 0	write 22 → Mem[0]	Mem[0] = 22
28	lw x7, 0(x0)	0 + 0	read Mem[0]	x7 = 22
32	beq x3, x7, 8	22 − 22 = 0	—	taken → PC = 40
36	addi x8, x0, 1	—	—	skipped
40	addi x9, x0, 99	0 + 99	—	x9 = 99
44	beq x0, x0, 0	0 − 0 = 0	—	halt loop

Chapter 8 — Simulation

Tools: Icarus Verilog 12 (compiler/simulator) and GTKWave (waveform viewer).

Testbench: generates a 10 ns clock, holds reset for one cycle, runs the processor for 15 cycles, prints the state every cycle, records all signals to `cpu.vcd`, and compares eleven values against hand-calculated expected results.

Commands:

```
iverilog -o cpu_sim program_counter.v instruction_memory.v register_file.v alu.v control_unit.v imm_gen.v data_memory.v
riscv_cpu.v testbench.v
vvp cpu_sim
gtkwave cpu.vcd
```

Result: all eleven checks passed (x1 = 12, x2 = 10, x3 = 22, x4 = 2, x5 = 8, x6 = 14, Mem[0] = 22, x7 = 22, x8 = 0, x9 = 99, PC = 44). The waveform confirms that the PC advances by 4 each cycle, jumps from 32 to 40 on the taken branch, and that register and memory writes occur only at rising clock edges.

Chapter 9 — Limitations

1. Only 8 of the ~40 RV32I instructions are supported (no JAL/JALR, BNE/BLT/BGE, LUI/AUIPC, shifts, SLT, XOR, byte/half-word loads and stores).
2. Other I-type arithmetic instructions (e.g. ANDI) are not distinguished; they would be executed as ADDI.
3. Invalid or unsupported instructions are not detected.
4. Instruction and data memories are small (64 words each) and the program is hard-coded in the instruction memory.
5. Single-cycle timing: clock period is set by the slowest instruction, so performance is low.
6. No pipelining, caches, interrupts, exceptions or privilege modes.
7. Verified in simulation only, not on physical hardware.

Chapter 10 — Future Scope

1. Implement the complete RV32I base instruction set so that compiled C programs can run.
2. Load programs from a file (`$readmemh`) instead of hard-coding them.
3. Convert the design into a 5-stage pipeline (IF, ID, EX, MEM, WB) with hazard detection and forwarding.
4. Add the M extension (multiply/divide).
5. Implement the design on an FPGA board and display register values on LEDs or a seven-segment display.

Chapter 11 — Viva Questions (Summary)

Question	Short answer
What is RISC-V?	An open, free, simple instruction set architecture.
What is single-cycle?	Every instruction finishes in one clock cycle (CPI = 1).
Role of the PC?	Holds the address of the current instruction.
Why PC + 4?	Each instruction is 4 bytes; memory is byte-addressed.
What is the register file?	32 registers, 2 read ports, 1 write port; x0 = 0.
What does the ALU do?	Add, subtract, AND, OR; outputs a zero flag.
What does the control unit do?	Decodes the opcode and generates control signals.
Why multiplexers?	To select data sources that differ between instructions.
What is an immediate?	A constant stored inside the instruction.
What is sign extension?	Copying the sign bit to widen a number without changing its value.
ADD?	Read rs1, rs2 → ALU adds → write rd → PC + 4.
LW?	ALU: rs1 + imm = address → read memory → write rd.
SW?	ALU: rs1 + imm = address → write rs2 to memory; no register write.
BEQ?	ALU subtracts; if zero, PC = PC + imm, else PC + 4.
Slowest instruction?	LW — it uses every unit.
Why separate instruction/data memory?	To fetch and access data in the same cycle.
Purpose of clock?	Synchronises updates of PC, registers and memory.
Purpose of reset?	Starts the processor from PC = 0.
Why Verilog?	Standard HDL; can be simulated and synthesised.
Is this complete RISC-V?	No, an 8-instruction subset of RV32I.
Future work?	Full RV32I, pipelining, FPGA implementation.

A full set of 54 questions with detailed answers and 14 follow-up questions is given in Part 11.

Conclusion

A single-cycle RISC-V processor supporting eight instructions was designed in Verilog and verified by simulation. The project demonstrates the complete instruction execution cycle — fetch, decode, execute, memory access, write back and next-PC selection — and the role of the control unit and multiplexers in steering data through a shared datapath.

References

1. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*, Morgan Kaufmann.
2. *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International.
3. S. Harris and D. Harris, *Digital Design and Computer Architecture, RISC-V Edition*, Morgan Kaufmann.
4. Icarus Verilog documentation; GTKWave user guide.